Quick Start & POC Guide

From Zero to Migrated VM in 30 Minutes

Step-by-step hands-on guide for IT teams. Install hyper2kvm, run system checks, convert a local VMDK, migrate from vSphere, test Windows VMs, deploy to KubeVirt, and validate with the POC checklist. Everything you need to prove the migration works before committing to a full project.

30 Minutes | 6 Labs | POC Checklist | Copy-Paste Commands | Proprietary (HyperSDK)

Lab 1: Install & System Check 5 min

Get hyper2kvm running on any Linux machine.

Prerequisites

Any Linux machine: Fedora 38+, RHEL 9+, Ubuntu 22.04+, Debian 12+. 4GB RAM minimum. 20GB free disk. Root or sudo access.

1 Install dependencies + hyper2kvm

```
# Option A: One-command quickstart (recommended)
sudo ./scripts/quickstart.sh

# Option B: Manual install
# Fedora / RHEL:
sudo dnf install -y qemu-img qemu-system-x86 libvirt virt-install python3-pip
# Ubuntu / Debian:
sudo apt install -y qemu-utils qemu-system-x86 libvirt-daemon-system python3-pip

# Install hyper2kvm
git clone https://github.com/ssahani/hyper2kvm.git && cd hyper2kvm
pip3 install -e .

# Install govc (for vSphere exports)
curl -L https://github.com/vmware/govmomi/releases/latest/download/govc_Linux_x86_64.tar.gz \
  | sudo tar xzf - -C /usr/local/bin govc

# Build zkvm TUI
cd zkvm && go build -o zkvm . && sudo cp zkvm /usr/bin/zkvm && cd ..
```

2 Run system check

```
# Validates 9 areas: KVM support, disk space, qemu-img, libvirt, govc, etc.
h2kvmctl doctor

# Expected output: all checks GREEN
# If any check fails, fix it before proceeding
```

3 Verify installation

```
h2kvmctl --version          # hyper2kvm version
govc version                # govc version
zkvm --help 2>&1 | head -1  # zkvm TUI available
virsh list --all            # libvirt running
kvm-ok 2>/dev/null || grep -c vmx /proc/cpuinfo  # KVM support
```

Checkpoint: All 5 commands above should succeed. If `h2kvmctl doctor` shows all green, you're ready for Lab 2.

Lab 2: Local VMDK Conversion 10 min

Convert a VMDK to QCOW2 and boot it on KVM — no vCenter needed.

4 Get a test VMDK

```
# Option A: Use any existing VMDK from your VMware environment
# Copy it to the local machine via SCP, USB, or datastore mount

# Option B: Download Photon OS (small, fast, perfect for testing)
curl -LO https://packages.vmware.com/photon/5.0/Rev1/ova/photon-5.0-ova.ova
tar xf photon-5.0-ova.ova   # Extracts .vmdk + .ovf
```

5 Create migration config

```
cat > local-test.yaml <<'EOF'
cmd: local
vmdk: ./photon-disk1.vmdk        # Path to your VMDK
output_dir: ./converted
out_format: qcow2
flatten: true
regen_initramfs: true             # Rebuild with VirtIO drivers
emit_domain_xml: true             # Generate libvirt XML
vm_name: photon-poc
memory: 1024
vcpus: 2
machine: q35
EOF
```

6 Run conversion

```
sudo h2kvmctl --config local-test.yaml
```

Watch the output: validate → flatten → convert → guest fix → generate XML → define VM.

7 Verify and boot

```
ls -lh converted/                    # QCOW2 file + domain XML created
qemu-img info converted/*.qcow2     # Confirm format: qcow2
virsh list --all                     # photon-poc should appear
virsh start photon-poc               # Boot the VM
virsh console photon-poc             # Connect to console (Ctrl+] to exit)
```

Success criteria: QCOW2 created, VM defined in libvirt, `virsh start` boots to login prompt. If you see the login screen, Lab 2 is complete.

Or Use the zkvm TUI (Interactive)

```
# Instead of YAML, use the guided TUI:
zkvm
# Tab 1 (Migration): set source file, output dir, deploy targets
# Press 1 to enable "Libvirt XML", then Ctrl+R to run
# Alt+4 (Libvirt tab): see your new VM, press 's' to start it
```

Lab 3: vSphere Migration 15 min

Connect to your vCenter and migrate a real VM.

8 Set vCenter credentials

```
export GOVC_URL="https://vcenter.yourdomain.com/sdk"
export GOVC_USERNAME="administrator@vsphere.local"
export GOVC_PASSWORD="your-password"
export GOVC_DATACENTER="your-datacenter"
export GOVC_INSECURE=1   # If using self-signed certificate
```

9 Discover VMs (read-only)

```
# CLI:
govc ls /$GOVC_DATACENTER/vm/
govc vm.info -json your-test-vm | jq '.virtualMachines[0].config.hardware'

# Or TUI (recommended):
zkvm
# Alt+2 → vSphere tab → VMs auto-discovered with CPU/RAM/disk/state
```

10 Migrate a test VM

```
cat > vsphere-test.yaml <<'EOF'
cmd: vsphere
vs_action: export_vm
vs_vm: your-test-vm              # Pick a non-production VM
output_dir: ./converted
out_format: qcow2
flatten: true
compress: true
regen_initramfs: true
emit_domain_xml: true
vm_name: migrated-test-vm
memory: 2048
vcpus: 2
machine: q35
```

```
boot_test: true                    # Auto-verify boot
EOF

sudo -E h2kvmctl --config vsphere-test.yaml
# -E preserves GOVC_* environment variables
```

11 Or use the TUI (zero YAML)

```
sudo -E zkvm
# Alt+2 → vSphere tab → connect → discover VMs
# Space to select your test VM → Ctrl+R to sync to Migration tab
# Review settings → Ctrl+R to run migration
# Alt+3 → Logs tab: watch progress in real-time
# Alt+4 → Libvirt tab: see your migrated VM, press 's' to start
```

Success criteria: VM exported from vSphere, converted, guest fixes applied, defined in libvirt, boot test passed. You should see "migrated-test-vm" in `virsh list --all`.

Lab 4: Windows VM Migration 15 min

Migrate a Windows Server VM with automatic VirtIO driver injection.

12 Get the VirtIO drivers ISO

```
# Fedora/RHEL:
sudo dnf install -y virtio-win
# ISO located at: /usr/share/virtio-win/virtio-win.iso

# Or download directly:
curl -LO https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso
```

13 Migrate Windows VM

```
cat > windows-test.yaml <<'EOF'
cmd: vsphere                        # Or: cmd: local + vhd: /path/to/disk.vhdx
vs_action: export_vm
vs_vm: your-windows-test-vm
output_dir: ./converted
out_format: qcow2
windows: true
guest_os: windows
virtio_win_iso: /usr/share/virtio-win/virtio-win.iso
win_stage: bootstrap
emit_domain_xml: true
vm_name: win-migrated
memory: 4096
vcpus: 4
machine: q35
uefi: true                          # Most Windows VMs use UEFI
boot_test: true
EOF

sudo -E h2kvmctl --config windows-test.yaml
```

What happens automatically: VirtIO storage driver injected (prevents BSOD), VirtIO network driver injected, VMware/Hyper-V tools removed, BCD updated, serial console enabled, boot verified (no BSOD).

Lab 5: KubeVirt Deployment 10 min

Deploy a migrated VM to Kubernetes via KubeVirt.

14 **Deploy to KubeVirt**

```
# If you have K3s / K8s + KubeVirt installed:
cat > kubevirt-test.yaml <<'EOF'
cmd: local
vmdk: ./photon-disk1.vmdk
output_dir: ./converted
out_format: qcow2
regen_initramfs: true
deploy_k8s: true                  # Generate KubeVirt manifest + apply
k8s_namespace: default
vm_name: photon-kubevirt
memory: 1024
vcpus: 2
EOF

sudo h2kvmctl --config kubevirt-test.yaml

# Verify:
kubectl get vm                    # VirtualMachine should appear
kubectl get vmi                   # VirtualMachineInstance running
virtctl console photon-kubevirt   # Connect to console
```

Don't have KubeVirt? Install K3s + KubeVirt in 5 minutes: `curl -sfL https://get.k3s.io | sh -` then install KubeVirt operator. Or skip Lab 5 — KVM deployment (Labs 2-4) proves the core migration works.

Lab 6: Hyper-V & Cloud Migration 10 min

Test VHD/VHDX conversion and cloud VM repatriation.

Hyper-V VHD/VHDX Conversion

15 Migrate from Hyper-V

```
# Copy VHD/VHDX from Hyper-V host to this machine
# Default Hyper-V path: C:\Users\Public\Documents\Hyper-V\Virtual Hard Disks\

cat > hyperv-test.yaml <<'EOF'
cmd: local
vhd: /exports/ubuntu-server.vhdx  # Your VHD/VHDX file
output_dir: ./converted
out_format: qcow2
flatten: true
regen_initramfs: true              # Swaps hv_storvsc for virtio_blk
emit_domain_xml: true
vm_name: from-hyperv
memory: 2048
vcpus: 2
machine: q35
boot_test: true
EOF

sudo h2kvmctl --config hyperv-test.yaml
```

Cloud VM Repatriation (AWS/Azure)

From AWS EC2

```
# 1. Create EBS snapshot
aws ec2 create-snapshot \
  --volume-id vol-xxx

# 2. Export to S3
aws ec2 export-image \
```

From Azure

```
# 1. Snapshot managed disk
az snapshot create \
  --name snap01 --source disk01

# 2. Generate SAS URL + download
az snapshot grant-access \
```

```
  --disk-image-format RAW \
  --s3-export-location ...

# 3. Download and convert
sudo h2kvmctl --cmd local \
  --raw ./aws-export.raw \
  --regen-initramfs \
  --emit-domain-xml
```

```
  --name snap01 --duration 3600
wget "SAS_URL" -O azure.vhd

# 3. Convert
sudo h2kvmctl --cmd local \
  --vhd ./azure.vhd \
  --regen-initramfs \
  --emit-domain-xml
```

POC Validation Checklist

Complete this checklist to confirm migration readiness. All items should pass before committing to a full project.

	Test	How to Verify	Expected Result	Lab
☐	**System check passes**	`h2kvmctl doctor`	All 9 checks green	Lab 1
☐	**VMDK → QCOW2**	`qemu-img info *.qcow2`	Format: qcow2, correct size	Lab 2
☐	**Linux VM boots on KVM**	`virsh start vm-name`	Reaches login prompt	Lab 2
☐	**fstab uses UUIDs**	`cat /etc/fstab` inside VM	UUID= entries, no /dev/sdX	Lab 2
☐	**initramfs has VirtIO**	`lsinitrd` inside VM	virtio_blk, virtio_net present	Lab 2
☐	**vSphere discovery works**	zkvm vSphere tab or `govc ls`	VMs listed with metadata	Lab 3
☐	**vSphere export works**	Run vsphere-test.yaml	VMDK downloaded + converted	Lab 3
☐	**Windows VM boots (no BSOD)**	`virsh start win-vm`	Reaches login screen	Lab 4
☐	**VirtIO drivers loaded (Windows)**	Device Manager inside VM	VirtIO SCSI + NetKVM visible	Lab 4
☐	**KubeVirt deploy works**	`kubectl get vm`	VM created, VirtualMachineInstance running	Lab 5
☐	**VHD/VHDX converts**	Run hyperv-test.yaml	QCOW2 created, VM boots	Lab 6
☐	**Network works post-migration**	`ping` from migrated VM	Connectivity OK	All labs
☐	**Boot test auto-verifies**	`boot_test: true` in YAML	Automated pass/fail reported	Lab 3, 4
☐	**YAML automation works**	`h2kvmctl --config test.yaml`	Reproducible — same config = same result	All labs
☐	**zkvm TUI usable**	Run `zkvm`, complete a migration	Guided flow, no CLI memorization	Lab 2, 3

Common Issues & Quick Fixes

Issue	Cause	Fix
govc: certificate error	vCenter uses self-signed cert	`export GOVC_INSECURE=1`

Permission denied on qcow2	KVM needs root for disk access	Run with `sudo` (or `sudo -E` for env vars)
VM boots to GRUB rescue	initramfs missing VirtIO drivers	Ensure `regen_initramfs: true` in config
No network after boot	libvirt default network not active	`virsh net-start default`
Windows BSOD (0x7B)	Missing VirtIO storage driver	Add `windows: true` + `virtio_win_iso` path
Disk space error	/tmp too small for conversion	`export TMPDIR=/large/disk/tmp` before running
govc: access denied	Insufficient vCenter permissions	Need read access to datastore + VM inventory

POC Complete — What's Next?

You've proven hyper2kvm works. Here's how to move to production.

Step 1: Full Assessment

- Connect to vCenter read-only
- Inventory all VMs (CPU, RAM, disk, OS)
- Classify by tier (dev/staging/prod)
- Identify Windows VMs needing VirtIO
- Map network dependencies
- Estimate timeline and savings
- **Deliverable:** Migration plan + savings report

Step 2: Phased Migration

- Phase 1: Dev/test VMs (1 week)
- Phase 2: Staging VMs (2 weeks)
- Phase 3: Production waves (4-8 weeks)
- Phase 4: Windows/AD/SQL (2-4 weeks)
- Phase 5: Decommission VMware
- YAML batch automation for scale
- **Typical:** 200 VMs in 12-20 weeks

Step 3: Operate & Optimize

- Set up Prometheus + Grafana monitoring
- Configure backup/DR (Ceph, Longhorn)
- Train team on KVM/libvirt admin
- Cancel VMware licenses
- Document savings for leadership
- Consider KubeVirt for cloud-native
- **Result:** 70-90% cost reduction

POC Results Summary (Fill In)

Metric	Your Result	Notes
VMs tested in POC	_____	
First-boot success rate	_____%	
Conversion time per VM	_____ min	
Total VMs in estate	_____	
Current annual VMware/cloud cost	$_____	
Estimated annual savings	$_____	Typically 70-90% of current cost
Estimated migration timeline	_____ weeks	~30-40 VMs/week/engineer
POC verdict	GO / NO-GO	

You've Proven It Works. Now Scale It.

hyper2kvm converted your VMs. They booted on KVM. The pipeline is automated.
The tool is licensed*. The savings start the day you cancel VMware licenses.

Take the POC results to leadership. Build the business case. Start the migration.